Technical report — `SafeSignal`

A deeper write-up than the README: what was built, how it works, and what the numbers say.

- **Track:** `Track 2B — SafeSignal`
- **Event:** Online
- **Team:** `SafeSignal` — Ileana Mazilu (solo)
- **Demo:** https://youtu.be/axOxy9aDvLo (about 100 s; recorded with `swiss-ai/apertus-8b-instruct`, see section 3)
- **Code:** https://github.com/iancuileana83-lab/safesignal-apertus (public, Apache-2.0)

1. Summary

Incomplete side-effect reports are common: the medicine, dose, dates or outcome are missing, and the reporter has to be asked again. SafeSignal sits at the moment of reporting. A person writes what happened in their own words, in Romanian, English, German, French or Italian. Apertus 1.5 extracts patient, suspect medicine, reaction, dates and outcome and must quote the words behind every field. Plain rules, not the model, then check completeness against the minimum of a valid report and triage the urgency of the wording, showing a rule id for each reason. A person edits the draft, confirms the review and downloads a structured report. Nothing is sent anywhere. Headline result so far: the multilingual rule packs classify 15 of 15 text-only urgency samples correctly, against 10 of 15 for the English-only pack, and Apertus 1.5 (70B, hosted) returned valid JSON on all 38 runs, found the medicine in 95%, detected the report language in 100% and, combined with the rules, gave the expected urgency in 100% of runs (section 5; fictional samples, so a design check, not validation).

2. Architecture

```
free text (ro/en/de/fr/it)
   -> Apertus 1.5 (OpenAI-compatible /v1/chat/completions)
        fields + exact quote for each + report language + follow-up questions (JSON)
   -> validation: legal values only, quote must occur in the text (accent-insensitive)
   -> rules (no model): completeness, urgency with rule ids   <- red-flag packs en/ro/de/fr/it
   -> human edits and ticks "reviewed" -> structured JSON draft (never sent)
```

- **Server** (`src/server.mjs`): static files, `/api/status`, `/api/extract`. Zero npm dependencies, Node 22 in the container. Report text is neither logged nor stored. A small in-memory rate limit protects the model key.
- **Client** (`src/public/`): plain ES modules; the same `rules.js` runs in the browser, the server, the tests and the benchmark, so the checks are identical everywhere. UI in Romanian and English (`i18n.js`).
- **Model access** (`src/llm.mjs`): one OpenAI-compatible client; only `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY` change between endpoints. The key stays on the server.

Target architecture (mandatory)

Fits (a) on-premise and (b) air-gapped by design; not demonstrated in this submission. (c) is not claimed.

- The application is one container with no runtime dependency except the model endpoint. Pointing `LLM_BASE_URL` at an Apertus weights server on the same host or network leaves no external connection at runtime.
- **What was actually run for the demo:** the hosted Public AI gateway (`https://api.publicai.co/v1`), which is an external hosted service and therefore *not* air-gapped or on-premise. The demo is honest about this: the page badge reads "Public AI gateway (hosted)".
- **Not done:** serving Apertus weights locally (no GPU was available in the build window), so latency, memory and quality of a local deployment are unmeasured. (`apertus-v1.5-8b` was the intended local model; the 70B model needs several GPUs.)
- **External dependencies.** Build time: Docker base image `node:22-alpine`. Runtime: the model endpoint only. There are no npm packages.

3. Use of Apertus

- **Model:** `swiss-ai/apertus-v1.5-70b` through the Public AI gateway (id confirmed with `GET /v1/models` on 2026-10-15). The gateway also lists `swiss-ai/apertus-v1.5-8b`, but calls to it failed with HTTP 404 ("Model 'alias-

apertus' not found", a gateway-side error), so **all measured results use the 70B model**. The 8B model is the realistic one for a local single-GPU deployment; it was not measured here.

- **Demo video and screenshots (15 Oct 2026) used a different model:** `swiss-ai/apertus-8b-instruct` (Apertus 1.0, 8B instruct, same gateway). At recording time the v1.5 models were unreliable on the free tier (the 70B answered about one call in three with HTTP 429 otherwise; the 8B v1.5 returned HTTP 404), so the recording used the model that answered consistently. The app shows the exact model id in its badge and provenance line, and the video captions name it. The measured results in section 5 are from `swiss-ai/apertus-v1.5-70b`, not from this model.
- **How it is used:** inference only, one call per report. No fine-tuning, adapters or quantisation by us.
- **Where it runs:** hosted by the Public AI inference utility (see section 2).
- **Prompt** (`src/llm.mjs`, `buildPrompt`): the report is wrapped between `<<<REPORT_TEXT` markers and declared to be data, never instructions. The model must return a single JSON object with, for each field, a `value` and an exact `quote` in the original language; enumerated fields take English values from a fixed list; free text stays in the language of the report; it also returns `language` (ISO 639-1), up to three follow-up questions and a one-sentence summary. Temperature 0, `response_format: json_object` with a retry without it if the server rejects it, one retry after HTTP 429/503, and a 60 s timeout.
- **Other models:** none. The development stand-in (`npm run standin`) is a regex script, not a model, and is labelled as such in the badge. No other open-weights model was used as a judge.
- **Why rules decide urgency:** a model asked to triage can be steered by text inside the report. Here the model only extracts; urgency comes from the rules, so the model cannot lower it. A Romanian and an English injection sample are part of the tests.

4. Data

- **Sample set:** 19 fictional reports in `data/cases.json` (8 English, 8 Romanian, 1 German, 1 French, 1 Italian) with gold answers for medicine, sex, outcome and urgency. The English samples come from the original SafeSignal repository; the others were written for this project with the AI coding assistant. No real person, patient or product report was used. Released under CDLA-Permissive-2.0.
- **No personal data, no human-subject data, no scraped data.** The app tells users not to type real details.
- **Gold answers** were assigned when the samples were written, by the same people (author and assistant) who wrote the rules. They are a design check, not an independent reference standard.

5. Evaluation

Task: (a) urgency from wording alone, (b) field extraction. Metrics: exact match against the gold answer. Everything below is produced by `make bench` (`src/tools/bench.mjs`); raw files are in `bench/results/`.

5a. Rules baseline (measured, no model). Urgency from the report text alone, English-only rule pack vs. all five packs. Scored on the 15 samples whose gold urgency comes from wording alone (emergency or routine).

Language	Samples	English-only pack correct	All packs correct
English	6	6	6
Romanian	6	3	6
German	1	0	1
French	1	0	1
Italian	1	1	1
All	15	**10**	**15**

Reading it correctly: the English-only pack passes the *routine* samples trivially (nothing fires). On the 5 non-English *emergency* samples it catches 0 of 5; the multilingual packs catch 5 of 5. The patterns were written by the same author who wrote the samples, so this shows the packs behave as designed, **not** clinical validity or generalisation to unseen text.

5b. Apertus extraction quality (measured through the gateway).

Measured on 2026-10-15 with `swiss-ai/apertus-v1.5-70b` via the Public AI gateway: 19 samples x 2 repeats = 38 runs (raw files in `bench/results/`). "Urgency" is the rules' result using the fields the model extracted, compared with the gold urgency.

Setup	Runs	JSON valid	Medicine	Sex	Outcome	Urgency	Language detected	Quotes found in text
Baseline: empty form, no tool	-	-	0%	0%	0%	79% (rules on text alone, no fields: routine unless red-flag wording)	-	-
Ours: all samples	38	100%	95%	79%	79%	100%	100%	90%
Ours: Romanian	16	100%	100%	63%	75%	100%	100%	88%
Ours: English	16	100%	88%	88%	75%	100%	100%	88%
Ours: German / French / Italian (1 each)	6	100%	100%	100%	100%	100%	100%	100%

Where it misses (looked at after the run; the gold answers were **not** changed to fit):

- **Sex and outcome left empty** where the gold answer is an inference ("her mum felt dizzy", "he is still in hospital"). The prompt says never to guess, so this is mostly the model following the instruction. One clear miss: Romanian "Dl K." (Mr K.) was not mapped to male.
- **Medicine** returned as "amoxicillin syrup" instead of "amoxicillin" (strict match; arguably acceptable).
- **Ungrounded quotes (10% of filled fields)** include a reporter role guessed from an altered quote ("He told me myself" for "He told me himself"). The grounding check flagged it, which is what it is for.
- Latency and throughput are **not reliably measured**: the gateway caches identical requests (about 200 ms) and answered concurrent requests on the free tier with HTTP 429. Observed fresh latency of the 70B model for one extraction: 8.4 s, 9.0 s and 9.5 s (about 950 prompt and 500 to 540 completion tokens, 57 to 60 tokens/s). See `bench/results/README.md`.

The baseline urgency figure is the rules applied to the text with no extracted fields, computed over the same 19 samples: 15 of 19 correct (79%). The 4 misses are the priority samples that depend on an extracted field (a child under 2 years, pregnancy), which the rules only see once the model has filled the fields. That is the part of the 100% that the model contributes.

6. Limitations

- Rules are keyword and pattern based, not clinical. They cannot understand negation ("no difficulty breathing") and will over-trigger there; this errs towards a human looking sooner, but it is a false alarm.
- The Romanian, German, French and Italian patterns and the Romanian interface texts were drafted with an AI coding assistant. **Romanian UI texts and rule patterns reviewed on 15 Oct 2026 by the author, a pharmacy assistant with about 20 years of experience in community pharmacies.** The German, French and Italian patterns have **not** been reviewed by a native speaker or a clinical reviewer.
- Only 19 fictional samples; one German, one French, one Italian. Dialects, abbreviations, typing without diacritics and brand names are barely covered. Accent-insensitive matching helps with missing diacritics.
- The extraction quality of the 8B model on real, messy Romanian is unknown beyond the fictional set.
- The hosted gateway is a third party: availability and rate limits are not under our control, and report text leaves the machine in the demo configuration. The demo says not to enter real data.
- The public endpoint spends the owner's gateway credit; only a per-address rate limit protects it.
- No accessibility audit, no regulatory validation, no real-world use.

7. Reproducibility

- **Runtime:** Docker and `make`. Node 22 inside the image; Node 18+ if run without Docker.
- **Run:** `make run` from the repository root (it calls `track_2b/Makefile`). Settings come from the environment (`LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`) or `track_2b/.env`.

- **Tests:** `make -C track_2b test` (20 tests: rules, multilingual urgency, accent-insensitive grounding, config).
- **Benchmark:** `make -C track_2b bench`. Without a model endpoint it still produces the rules baseline.
- **Seeds:** none; temperature is 0, but a hosted model gives no determinism guarantee.
- **Commit:** `0610d99` is the last commit that changed code, tests or CI. Later commits only fill in the links of this report and regenerate its PDF. The CI workflow `Docker build and make run (no API key)` passed on that commit (clean checkout, no key).

8. Next steps

- Review of the Romanian (and other) patterns and texts by pharmacovigilance professionals; a larger, independently labelled multilingual set (published as a Hugging Face dataset).
- Negation handling and a model-assisted second opinion that can only *raise* urgency, never lower it.
- Serve Apertus 8B locally (vLLM or llama.cpp) and measure an air-gapped deployment end to end.
- Voice input for Romanian-speaking pharmacy staff; mapping the draft to a regulator's reporting format, still with a human before any submission.

Origin and disclosure

- **Base application.** SafeSignal was first built for the AMD Developer Hackathon: ACT III with a different model (Gemma, planned on AMD Instinct hardware), in the separate repository https://github.com/iancuileana83-lab/safesignal. Its first commit is dated 2026-10-13, inside the Hack Apertus period (1–16 October 2026).
- **New in this repository:** Apertus 1.5 as the model; Romanian-first multilingual rule packs (en/ro/de/fr/it) with accent-insensitive matching; Romanian and English UI; report-language field; per-language quality and rules-baseline benchmark; Public AI gateway support (User-Agent, retry); Docker and `make run`; Apache-2.0 licence. **Removed:** the AMD replay and recording machinery and all AMD wording.
- **AI assistance.** The code, tests, rule patterns and texts were developed with an AI coding assistant (Claude Code) under the author's direction. The author is responsible for the submission.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Hack Apertus project template and Track 2B README: https://github.com/HackApertus/project-template
- Public AI API documentation: https://platform.publicai.co/docs
- Apertus 1.5 models: https://huggingface.co/swiss-ai/Apertus-v1.5-8B